جامعة الفيصل
Alfaisal University

كلية الهندسة والحوسبة المتقدمة
College of Engineering and Advanced Computing

Introduction to Artificial Intelligence (SE 444)

Lecture 6: Constraint Satisfaction Problems

Prof. Anis Koubaa

akoubaa@alfaisal.edu

This lecture introduces Constraint Satisfaction Problems (CSPs) as an important problem-solving framework in Artificial Intelligence. You'll learn how AI agents tackle problems defined by variables and constraints, exploring fundamental techniques such as **backtracking search** and **constraint propagation**. We will also examine various **real-world applications** of CSPs, equipping you with the skills to model and solve complex challenges efficiently.



Outline

Constraint Satisfaction Problem (CSP)

Understanding the fundamental concepts and formalization of CSPs

CSP as a Standard Search Problem

Translating CSPs into the standard search framework

Backtracking Search

Core algorithm for systematically exploring solution space

Forward Checking

Pruning technique to detect failures early

Constraint Propagation

Advanced techniques for reducing search complexity

Arc Consistency

Maintaining consistency between variable domains

Local Search for CSPs

Alternative approaches using iterative improvement

Constraint Satisfaction Problems (CSPs)

Variables

Set of variables $X_1, X_2, \ldots, X_n$ that need values assigned

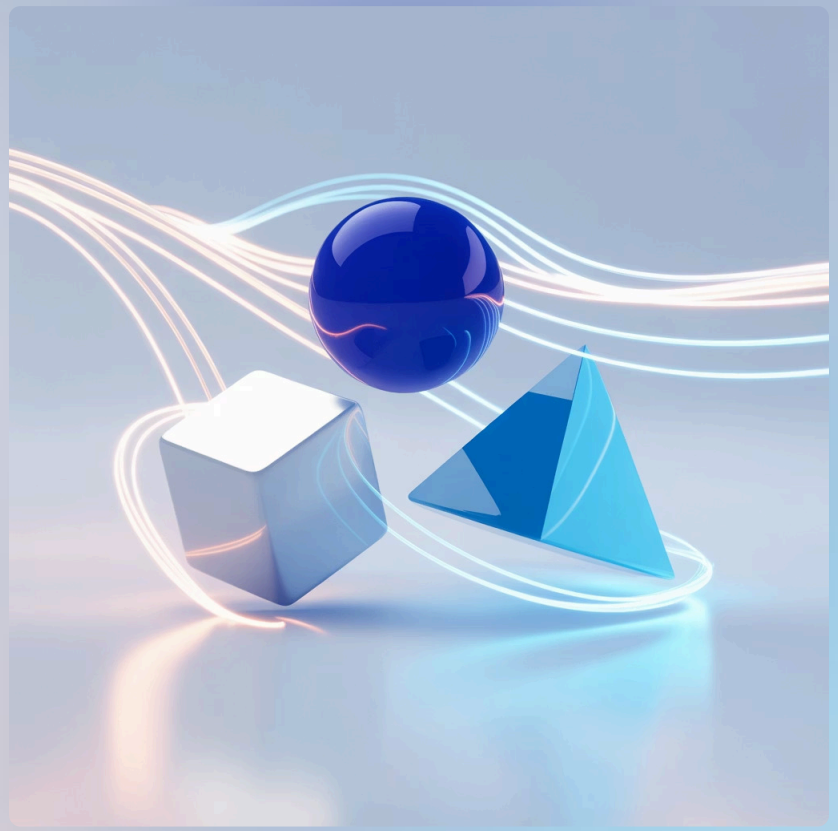
Domains

Each variable X_i has a domain D_i of possible values

Constraints

The rules that limit which combinations of values are allowed.

Important: Variables can have no value assigned (unassigned state) during the search process!



Example: Map Coloring (Graph Coloring)

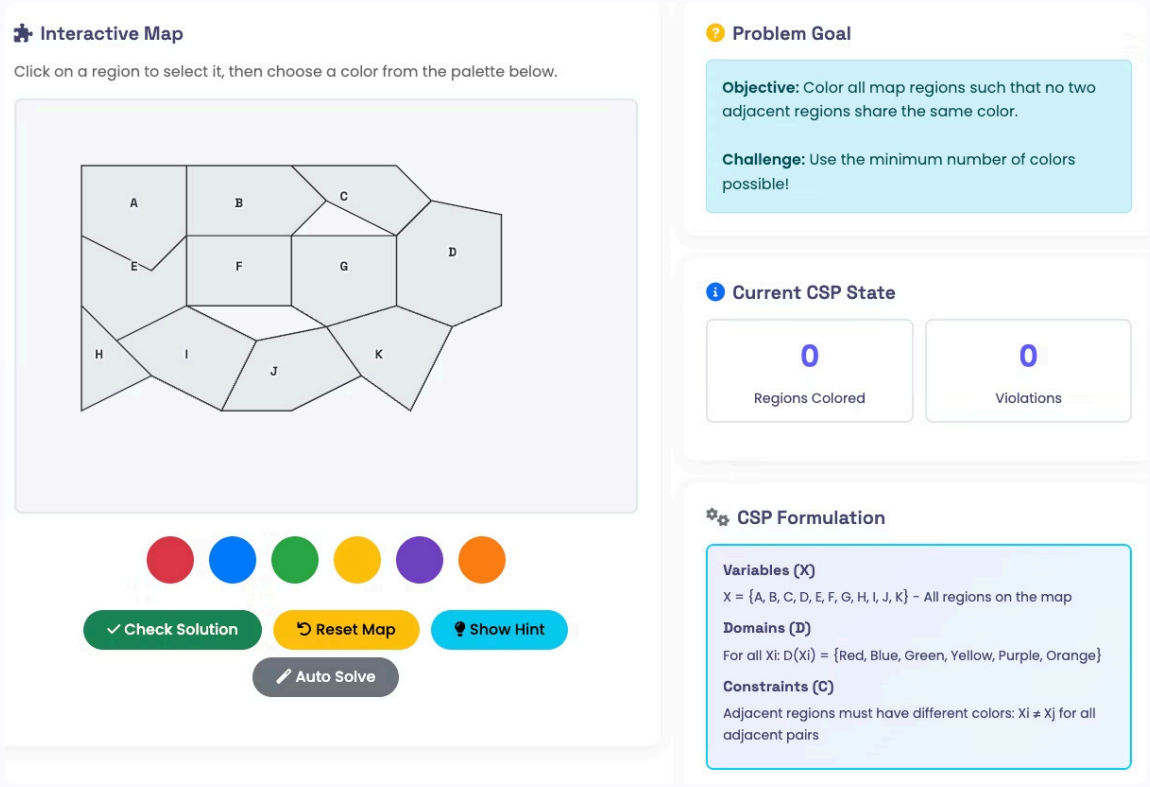
Problem Statement

Color adjacent regions of a map with different colors using the minimum number of colors possible.

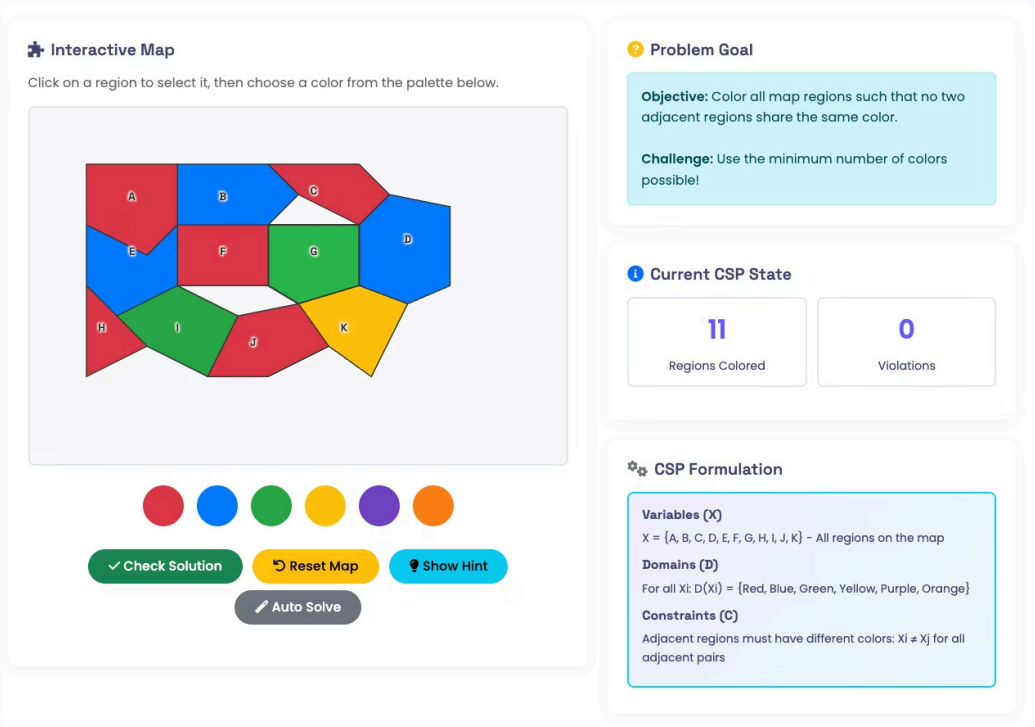
- **Variables (X):** {A, B, C, D, E, F, G, H, I, J, K} - All regions on the map
- **Domains (D):** For all Xi: D(Xi) = {Red, Blue, Green, Yellow, Purple, Orange}
- **Constraints (C):** Adjacent regions must have different colors: Xi ≠ Xj for all adjacent pairs

Constraint Graph

Each edge represents a constraint between neighboring regions that cannot share the same color.



Example: Map Coloring Solution



Valid Solution

- **Valid Solution:** A solution is a **complete and consistent assignment** of values to all variables.
- For example, in map coloring, a valid solution ensures that **no two adjacent regions share the same color**.

University Course Scheduling

Problem Description

University Course Scheduling aims to create a conflict-free timetable by assigning courses to specific time slots, rooms, and instructors, while satisfying various restrictions and optimizing resource utilization.

CSP Formulation

Variables (X)

X = {CS101-A, CS101-B, MATH101, CS201, PHYS101, MATH201} - Course sections to schedule

Domains (D)

D(Xi) = {Mon-9, Tue-9, ..., Thu-13} - Available room-time slots (13 total)

Constraints (C)

- **Instructor Conflict:** Dr. Smith cannot teach both CS sections simultaneously
- **Room Capacity:** Large lectures need big rooms (A101=100, others=50 seats)
- **Instructor Availability:** Dr. Brown unavailable Friday afternoons



Example: N-Queens Problem

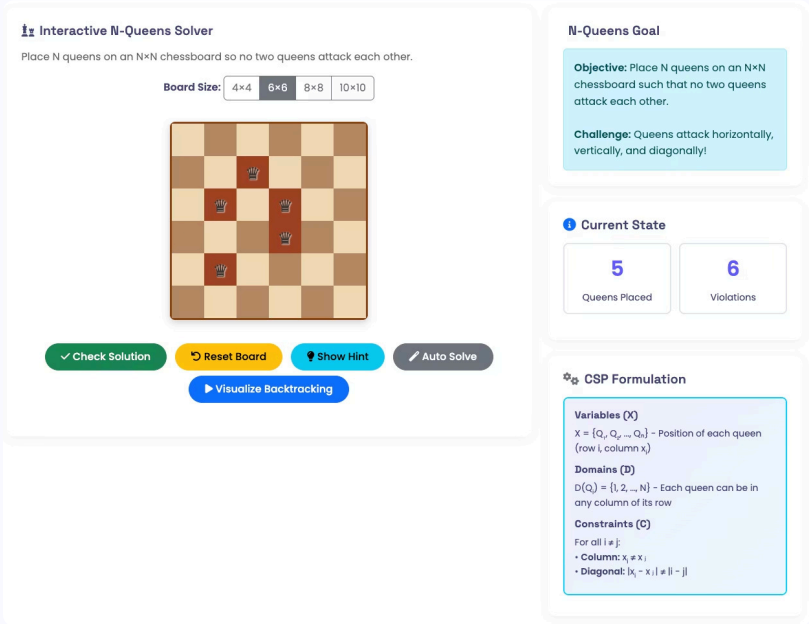
Problem Definition

Place N queens on an N×N chessboard such that no two queens attack each other. Queens can attack horizontally, vertically, and diagonally.

Formulation 1: Standard (N^2 Variables)

- **Variables:** X_{ij} for each square (i,j) on the N×N board
- **Domain:** {0, 1} (where 0 = empty square, 1 = queen)
- **Constraints:** Ensure no two queens are in the same row, column, or diagonal.

This formulation leads to N^2 variables, resulting in complex constraints and a large search space.



The image illustrates the N-Queens problem on a chessboard. The X_{ij} notation refers to the cell (row i, column j).

Example: N-Queens Problem

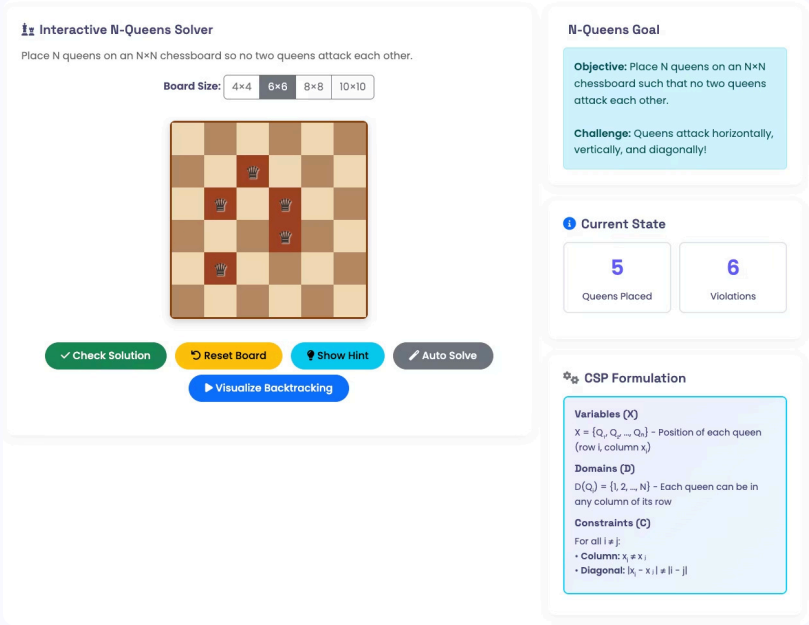
Problem Definition

Place N queens on an N×N chessboard such that no two queens attack each other. Queens can attack horizontally, vertically, and diagonally.

Formulation 2: Reduced (N Variables)

- **Variables (Q):** $Q_1, Q_2, \ldots, Q_n$ representing the column position of each queen.
- **Domains (D):** $D(Q_i) = \{1, 2, \ldots, N\}$ for each queen, meaning each queen can be in any column of its respective row.
- **Constraints (C):** For any two distinct queens Q_i and Q_j (where $i \neq j$):
    - **Column Constraint:** The queens must be in different columns: $Q_i \neq Q_j$
    - **Diagonal Constraint:** The queens must not be on the same diagonal: $|Q_i - Q_j| \neq |i - j|$

This alternative formulation significantly reduces the number of variables to N, simplifying the constraint satisfaction problem.



The image illustrates the N-Queens problem on a chessboard. The X_{ij} notation refers to the cell (row i, column j).

Example: Cryptarithmetic Puzzle

Variables

Letters: T, W, O, F, U, R

Carry digits: X_1, X_2

Each letter represents a unique digit, and carry variables handle arithmetic overflow.

Domains

Letters: {0, 1, 2, ..., 9}

Carries: {0, 1}

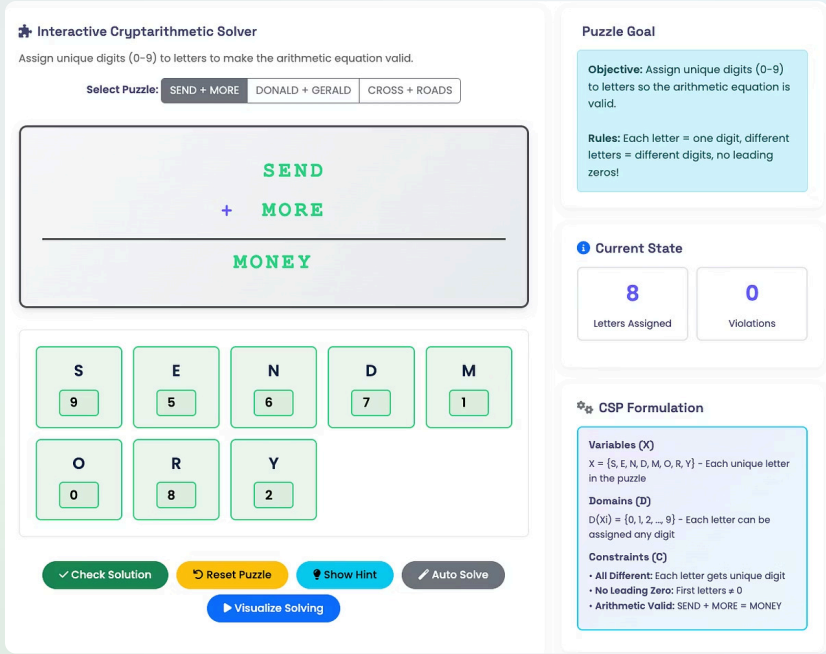
All letters must be assigned different digits (Alldiff constraint).

Constraints

Uniqueness: Alldiff(T, W, O, F, U, R)

Arithmetic:

- $O + O = R + 10 \times X_1$
- $W + W + X_1 = U + 10 \times X_2$
- $T + T + X_2 = O + 10 \times F$



Example: Sudoku

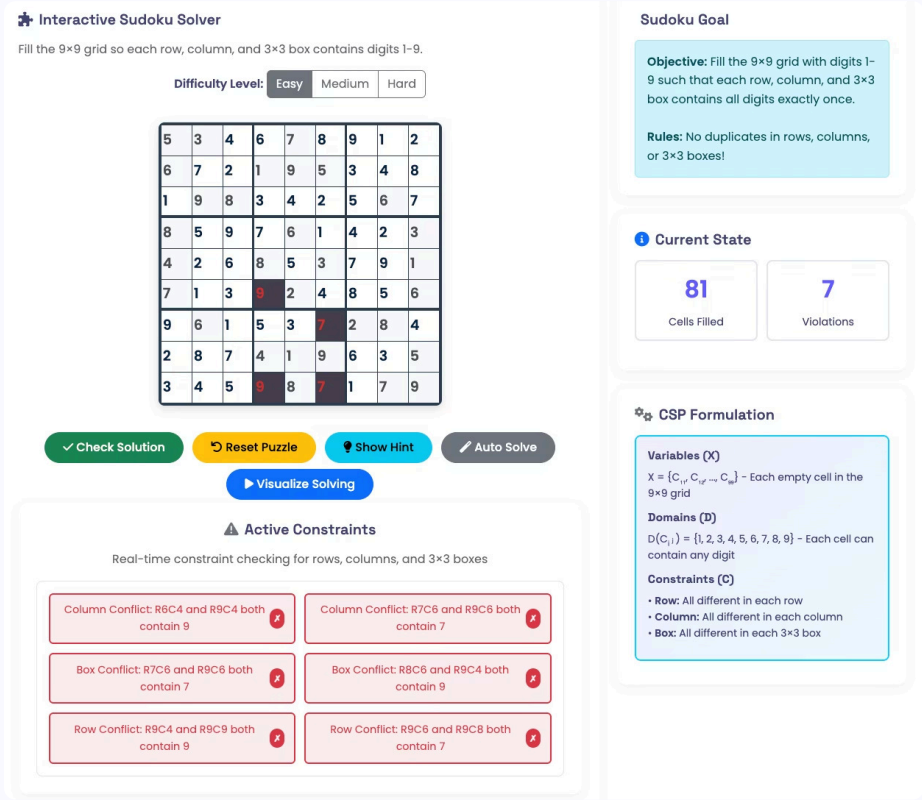
Problem Structure

Fill a 9×9 grid with digits 1-9 such that each row, column, and 3×3 subgrid contains all digits exactly once.

CSP Formulation:

- **Variables:** X_{ij} for each cell (i,j) in the grid → **81 variables** total.
- **Domain:** {1, 2, 3, 4, 5, 6, 7, 8, 9} for each empty cell
- **Constraints:**
  - Alldiff(X_{ij} in the same row)
  - Alldiff(X_{ij} in the same column)
  - Alldiff(X_{ij} in the same 3×3 unit)

Total of 81 variables with up to 9 values each, constrained by 27 Alldiff constraints.



Variable X_{ij} represents the digit in row i, column j



Real-World CSP Applications

Assignment Problems

Determining who teaches what class with fixed schedules. Key constraint: no teacher can be in two places simultaneously. Variables represent teacher-class assignments, domains are possible teachers, constraints ensure feasibility.

Timetabling Problems

Scheduling which classes occur when and where. Core constraint: no room double-booking. Variables represent time-class-room combinations, domains are available slots, constraints prevent conflicts.

Production Scheduling

Ordering production steps and transportation logistics. Constraints include resource availability, temporal dependencies, and capacity limits. Critical for manufacturing efficiency.

General Problem Solving

Many computational problems can be naturally formulated as CSPs, providing a unified framework for systematic solution approaches across diverse domains.

CSP as a Standard Search Formulation

State

Represents the **current partial assignment** of values to variables.

Includes which variables have been assigned and their current values.

Initial State

The **empty assignment { }** where all variables remain unassigned.

This is our starting point for the search process.

Successor Function

Choose an **unassigned variable** and assign it a legal value that doesn't violate any constraints.

If no legal assignment exists, backtrack (fail).

Goal Test

Check if we have a **complete and consistent assignment** - *all variables assigned with no constraint violations.*

This formulation allows us to apply standard search algorithms like depth-first search to systematically explore the solution space.

Backtracking Search

Key Insight: Variable Assignment Commutativity

In CSPs, the **order of variable assignments doesn't matter** for the final solution. For example, `[A = red then B = green]` produces the same result as `[B = green then A = red]`.

Search Tree Structure

Build a **search tree** where each level assigns a value **to one variable**

Tree Depth = n

Depth equals the number of variables in the problem

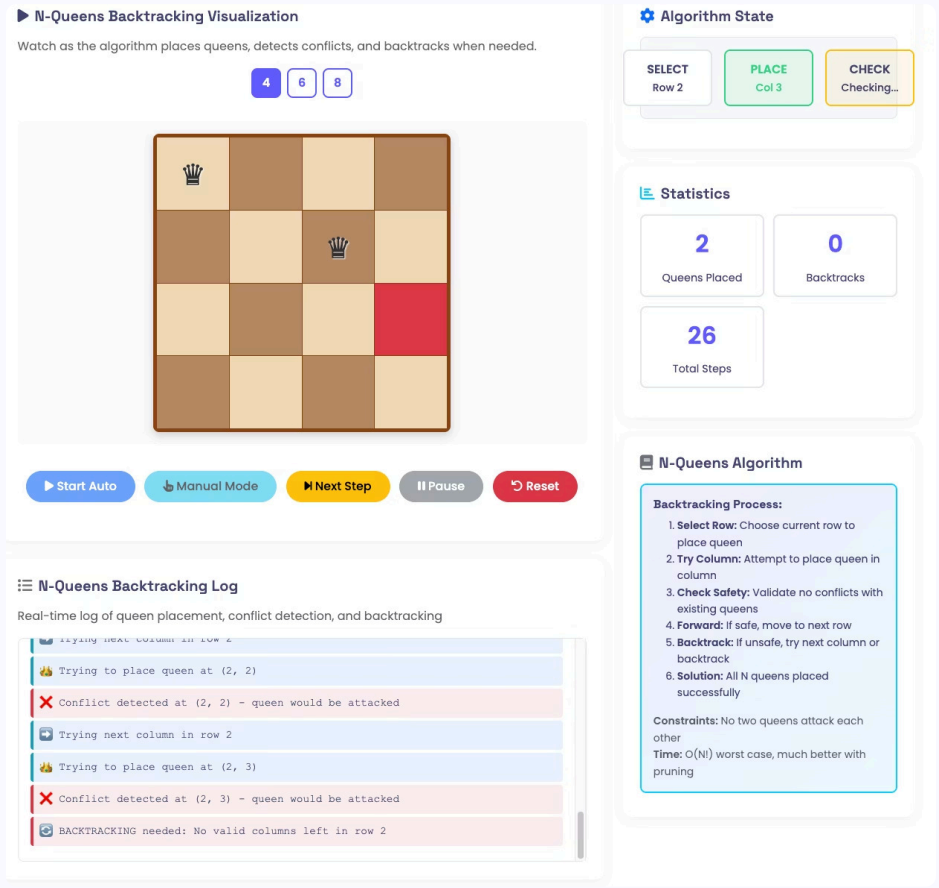
Leaves = d^n

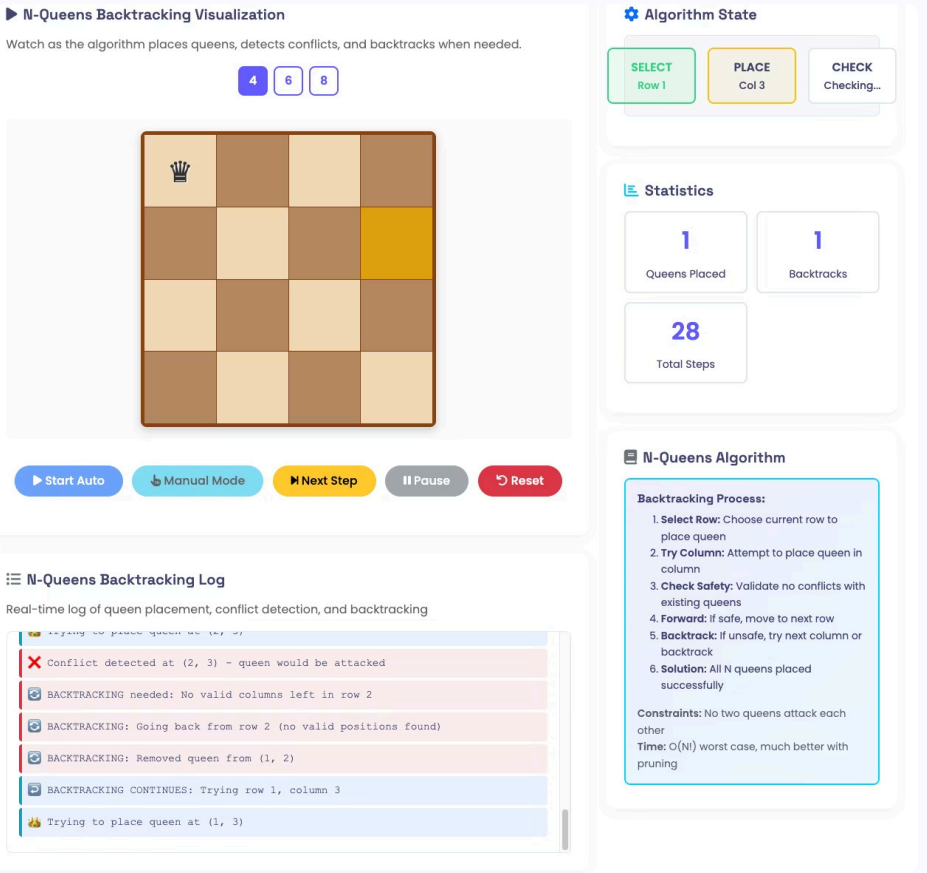
Number of leaf nodes equals **domain size** raised to variable count

Backtracking search is simply **depth-first search** applied to **CSPs** with **single-variable assignments** per level. This *systematic* approach explores all possible combinations while allowing efficient **pruning** of invalid paths.



Backtracking Example

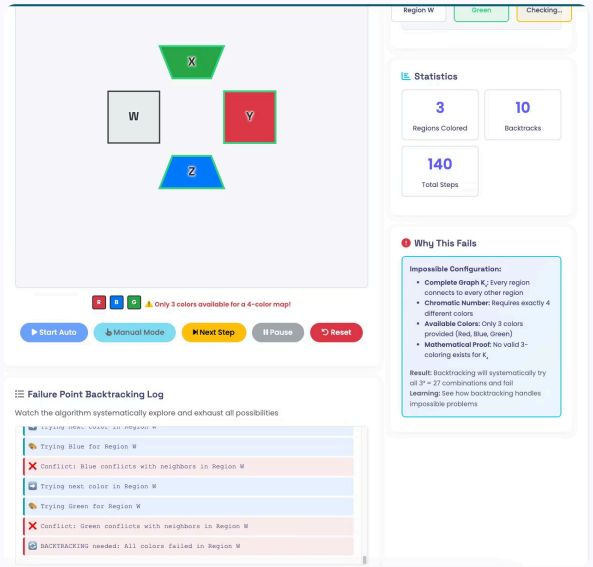




Backtracking Search (DFS)

Search Process Visualization

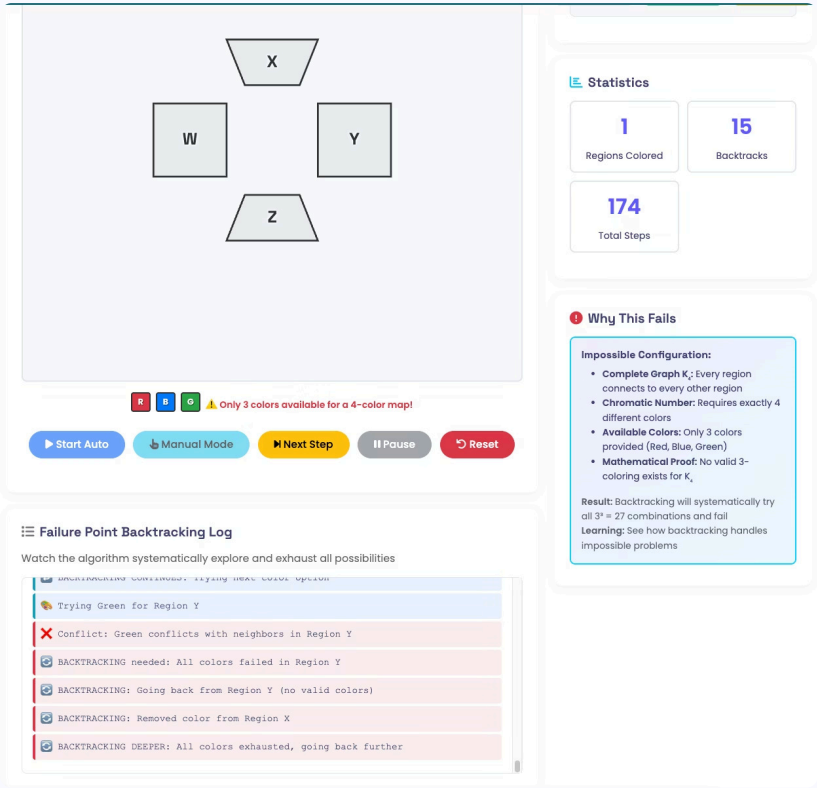
- 🌳 **Search Tree**: Represents **systematic** exploration of **variable assignments**.
- 📊 **Levels**: Each level corresponds to assigning a value to one variable (e.g., **WA → NT → Q → ...**).
- 🚫 **Constraint Violation**: If a **partial assignment** breaks a constraint, the algorithm **backtracks** to the previous level.
- 🔄 **Backtracking**: Tries the **next available value** for the variable.
- ✂️ **Pruning**: Backtracking **eliminates invalid branches early**, avoiding exploration of impossible paths.



Failure point - when no legal values remain

❌ Failure Point in Backtracking

- 🕸️ **Impossible Configuration**: The graph is a **complete graph (K_4)** → every region is connected to all others.
- 🔢 **Chromatic Number**: K_4 requires **4 different colors**.
- 🎨 **Available Colors**: Only **3 colors** provided (Red, Blue, Green).
- 🧱 **Mathematical Proof**: No valid **3-coloring** exists for K_4.
- 🔄 **Backtracking Process**:
- Systematically tries all $34 = 813^4 = 8134$-81 assignments (or fewer with pruning).
- Every branch eventually fails due to **constraint violation**.
- Ends with conclusion: ❌ *No solution exists under given constraints.*
- 📌 **Key Insight**: Backtracking not only finds solutions–it can also **prove impossibility**.



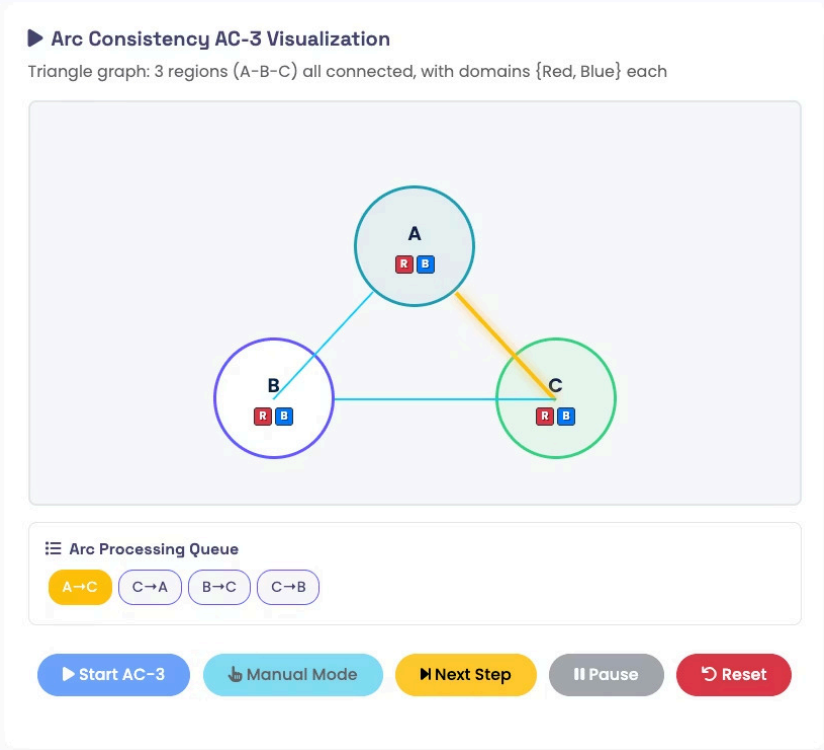
Arc Consistency (AC-3) in CSP

1. **What Arc Consistency Means:**
   - For every arc (X → Y), **every value in X's domain must have at least one supporting value in Y's domain**.
   - If a value in X has **no support** in Y, we remove it from X's domain.
   - If any domain becomes empty → ❌ no solution exists.
2. **The Queue of Arcs:**
   - AC-3 starts with a queue of **all arcs in both directions** (X → Y and Y → X for every constraint).
   - This ensures consistency is checked both ways.
3. **Processing Arcs:**
   - Take an arc (X → Y) from the queue.
   - For each value in X's domain, check if there is at least one supporting value in Y's domain.
   - If a value in X's domain has no support → remove it.
1. **Propagation:**
   - If X's domain changes, then all arcs of the form (Z → X) must be re-added to the queue.
   - This is because reducing X's domain might affect consistency with its neighbors.
2. **Termination:**
   - The algorithm continues until the queue is empty.
   - At that point:
     - If no domain is empty → the CSP is arc-consistent.
     - If any domain is empty → the CSP has **no solution**.

Arc Consistency (AC-3) in CSP

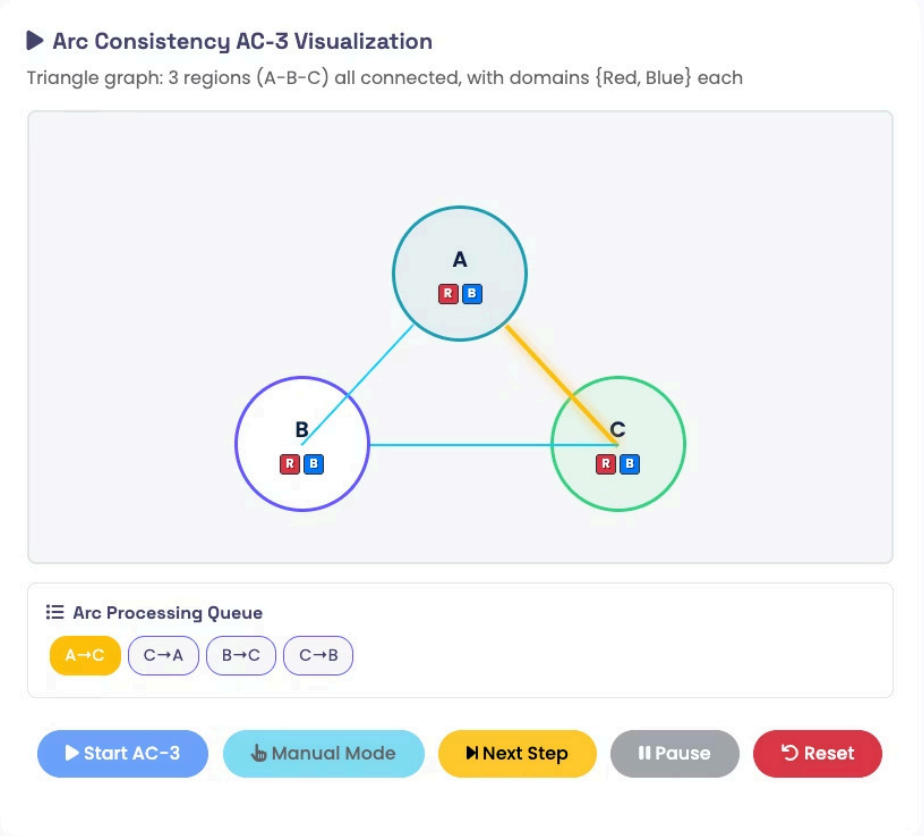
Introduction to Arc Consistency (AC-3)

- 🧩 **Constraint Satisfaction Problems (CSPs)**
  - Variables with **domains** of possible values
  - Constraints restrict which combinations are valid
- 🎯 **Goal of Arc Consistency**
  - Ensure that for every variable and every value, there is a **consistent value** in its neighbor's domain
- 🔄 **How AC-3 Works**
  a. Initialize: Add all arcs (X → Y) to a queue
  b. Process: For each value in X's domain, check support in Y
  c. Eliminate: Remove unsupported values from X
  d. Propagate: If X changes, recheck its neighbors
- 📉 **Benefit**
  - Reduces **search space** before backtracking
  - Detects some **inconsistencies early**
- ⚠️ **Limitation**
  - Arc consistency ≠ guaranteed solvability
  - CSP can still fail globally (e.g., triangle map with 2 colors)



Arc Consistency (AC-3) – Support Checking

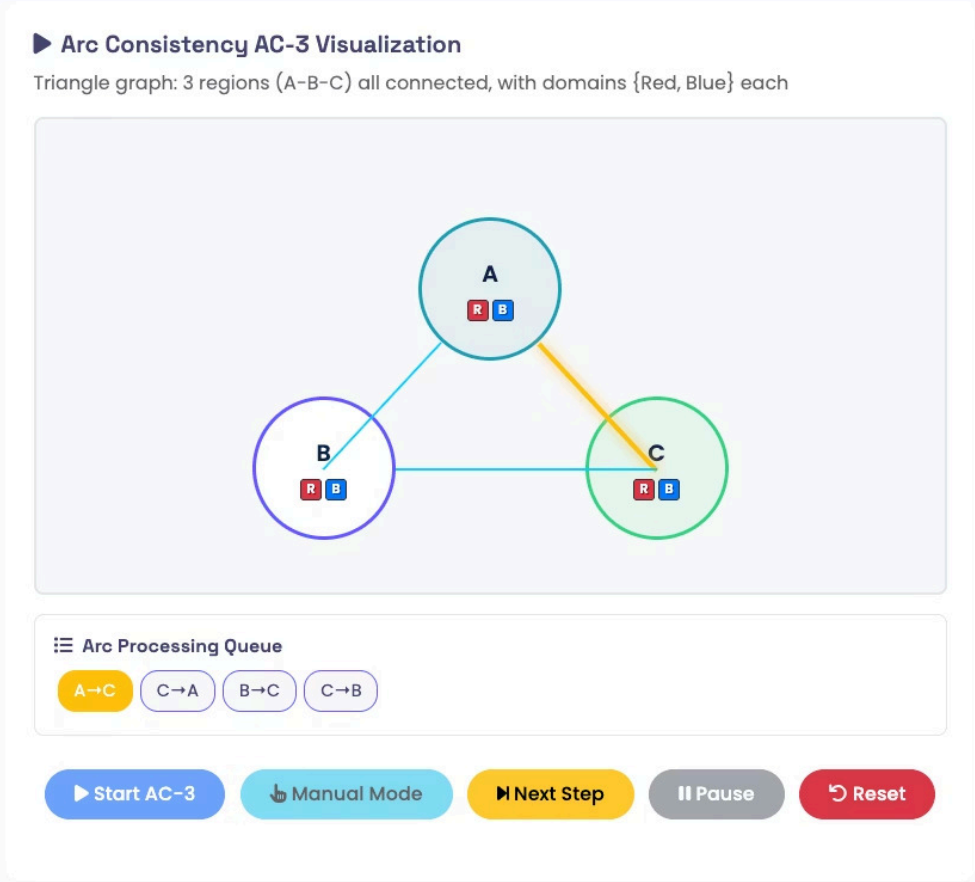
- 🟦 **Arc (X → Y):** For each value in **X's domain**, check if there's a **supporting value** in **Y's domain** that satisfies the constraint.
- ✅ **Supported Value:**
- Example: If **A = Red**, and **B's domain = {Red, Blue}**,
- B = Blue provides support (since A ≠ B).
- ❌ **Unsupported Value:**
- If no value in Y's domain satisfies the constraint,
- remove that value from X's domain.
- 🔄 **Propagation:**
- When a domain is reduced, re-add related arcs (Z, X) to the queue.
- Ensures constraint updates cascade through the graph.
- 📌 **In this case:**
- All values in A, B, C are supported.
- No eliminations occur → domains remain {Red, Blue}.
- ⚠️ **Key Insight:**
- Arc consistency ensures **local consistency** only.
- It does **not guarantee global solvability** (triangle needs 3 colors).



Why AC-3 is Used

🎯 Why AC-3 is Used?

- 🔄 **Constraint Propagation:**
  - Removes impossible values *before* search.
  - Shrinks domains early → fewer choices to explore.
- ⏳ **Efficiency Boost for Backtracking:**
  - **Without AC-3:** backtracking explores many dead ends.
  - **With AC-3:** many of those dead ends are pruned *in advance*.
- 🚨 **Early Failure Detection:**
  - If any variable's domain becomes empty → we know immediately that the CSP has **no solution**.
  - Saves time by avoiding pointless search.
- 🧩 **Practical Applications:**
  - **Sudoku:** removes digits from cells based on row/col/box constraints.
  - **Scheduling:** propagates conflicts (e.g., if Prof. A already assigned to 10am, remove 10am from all their other classes).
  - **N-Queens:** placing one queen automatically eliminates rows, columns, diagonals.



Why AC-3 is Used

Forward Checking

🧩 Definition of Forward Checking

Forward Checking is a technique used in solving **Constraint Satisfaction Problems (CSPs)** that helps prune the search space during **backtracking search**.

👉 **Definition:**
When a variable is assigned a value, forward checking **immediately eliminates inconsistent values** from the domains of all unassigned neighboring variables.

🎯 Why Forward Checking is Useful

- ✅ Detects failure **earlier** than plain backtracking.
- ✅ Reduces the search space by pruning inconsistent values.
- ✅ Works well with heuristics like **MRV** (Minimum Remaining Values), since domains shrink dynamically.

⚙️ How it Works (Step by Step)

1. **Assign a Value:**
   Pick a variable and assign it a value (e.g., in map coloring, set *WA = Red*).
2. **Look Ahead:**
   For each neighboring variable that shares a constraint with the assigned variable, check its domain.
3. **Eliminate Values:**
   Remove any values from the neighbor's domain that conflict with the assignment.
   - Example: If *WA = Red*, then remove "Red" from NT and SA's domains (since neighbors can't have the same color).
4. **Check for Failure:**
   If any neighbor's domain becomes empty → this indicates **failure**.
   - The algorithm backtracks immediately instead of going deeper into an impossible branch.

💡 **Summary:**
Forward Checking = *"look ahead"* pruning: after each assignment, update the remaining domains by removing impossible values. If a domain becomes empty → fail early.

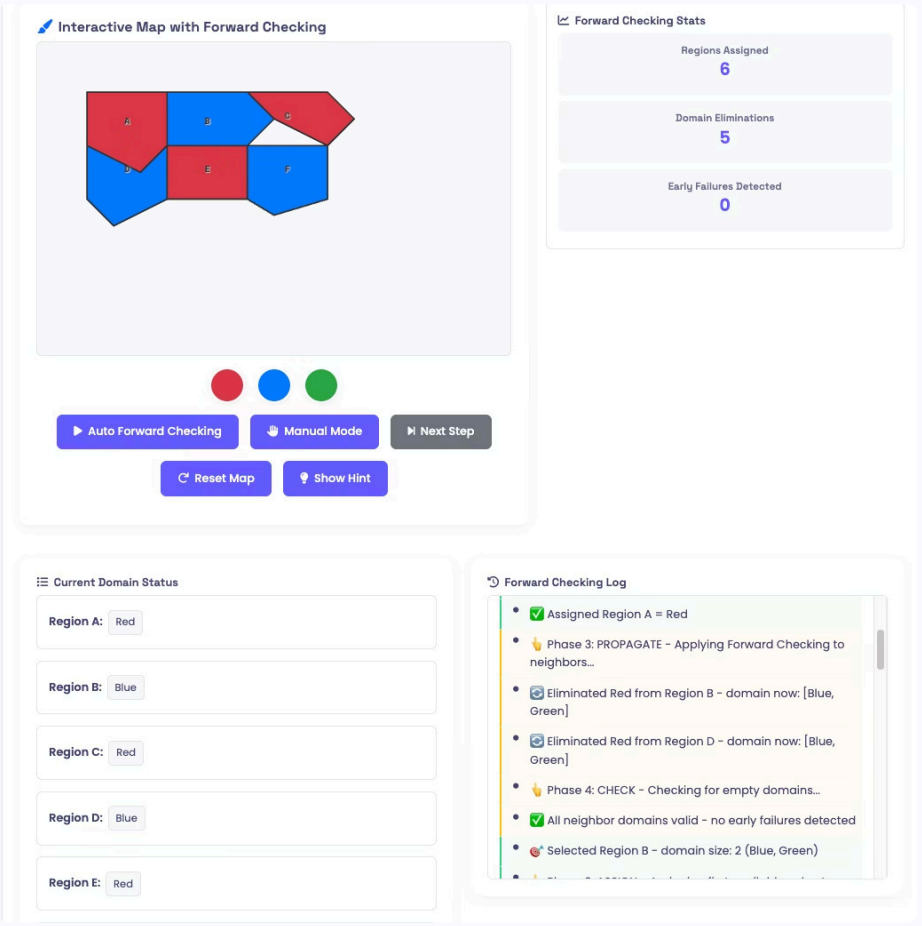
Forward Checking - Concept

Core Idea

Keep track of the **remaining legal values** for all unassigned variables after each assignment.
If any variable's domain becomes empty, **immediately backtrack**.

Key Benefits

- **Early failure detection**
  Spot **dead-ends quickly** without fully exploring **unworkable** branches.
- **Domain reduction**
  Continuously **eliminate impossible values** to shrink the search space.
- **Informed decisions**
  Always have up-to-date domain sizes, making heuristics like **MRV** (Minimum Remaining Values) more effective.



Forward checking transforms CSP search from blind exploration to informed constraint propagation, significantly reducing the search space in most practical problems.



Backtracking Search Algorithm

Improving Backtracking Efficiency

Variable Selection

Which variable should be assigned next? Smart ordering can dramatically reduce the search space by making better choices early.

Value Ordering

In what order should we try values for the chosen variable? Good value ordering can find solutions faster.

Early Failure Detection

Can we detect inevitable failure before fully exploring a branch? **Pruning** impossible paths saves significant computation.

These three optimization strategies form the foundation for efficient CSP solving. The basic backtracking algorithm can be exponentially slow, but with these enhancements, many real-world problems become tractable.

Variable Ordering vs. Value Ordering

These two heuristics are critical for optimizing backtracking search in Constraint Satisfaction Problems (CSPs).

1. **Variable Ordering**

👉 Decides **which variable to assign next** during search.
"Who goes first?"

- **Goal:** Minimize the chance of exploring dead ends.
- **Strategies (Heuristics):**
  - **MRV (Minimum Remaining Values):** Choose the variable with the fewest legal values left.
  - **Degree Heuristic:** Choose the variable that is involved in the largest number of constraints with unassigned variables.

📌 Example: In **Sudoku**, pick the cell that has the fewest candidate numbers.

2. **Value Ordering**

👉 Decides **which value to try first** for the chosen variable.
Which choice do we try first? 🎨

- **Goal:** Maximize the chance of success by keeping the problem flexible.
- **Strategies (Heuristics):**
  - **LCV (Least Constraining Value):** Choose the value that rules out the fewest options for neighboring variables.

📌 Example: In **map coloring**, if a country can be {Red, Green}, and choosing Red blocks more options for neighbors, pick Green first.

⚖️ **Key Difference:** Variable Ordering answers "Who goes first?" while Value Ordering answers "Which choice do we try first?". They work together: first select the variable, then select a value for it.

Variable and Value Ordering Heuristics

VARIABLE ORDERING

Most Constrained Variable

Strategy: Choose the variable with the **fewest** remaining legal values

Also called: **Minimum Remaining Values (MRV)** heuristic

Rationale: Variables with fewer options are more likely to cause failures, so handle them early to detect problems quickly.

VALUE ORDERING

Least Constraining Value

Strategy: Choose the value that **eliminates the fewest** options for neighboring variables.

Rationale: Preserve **maximum flexibility** for future assignments, making it more likely to find a complete solution.

These heuristics work together: **MRV** for variable selection focuses on failure detection, while **least constraining value** for value selection focuses on solution finding.

Minimum Remaining Values (MRV)

The "Fail-Fast" Principle

MRV is a **heuristic** for selecting *which variable to assign next*. It's also known as the **Most Constrained Variable (MCV)** heuristic.

How it works:

1. Keep track of **remaining legal values** for each unassigned variable
2. Choose the variable with the **smallest domain** (fewest legal values)
3. If a variable **has zero legal values**, immediately detect **failure**

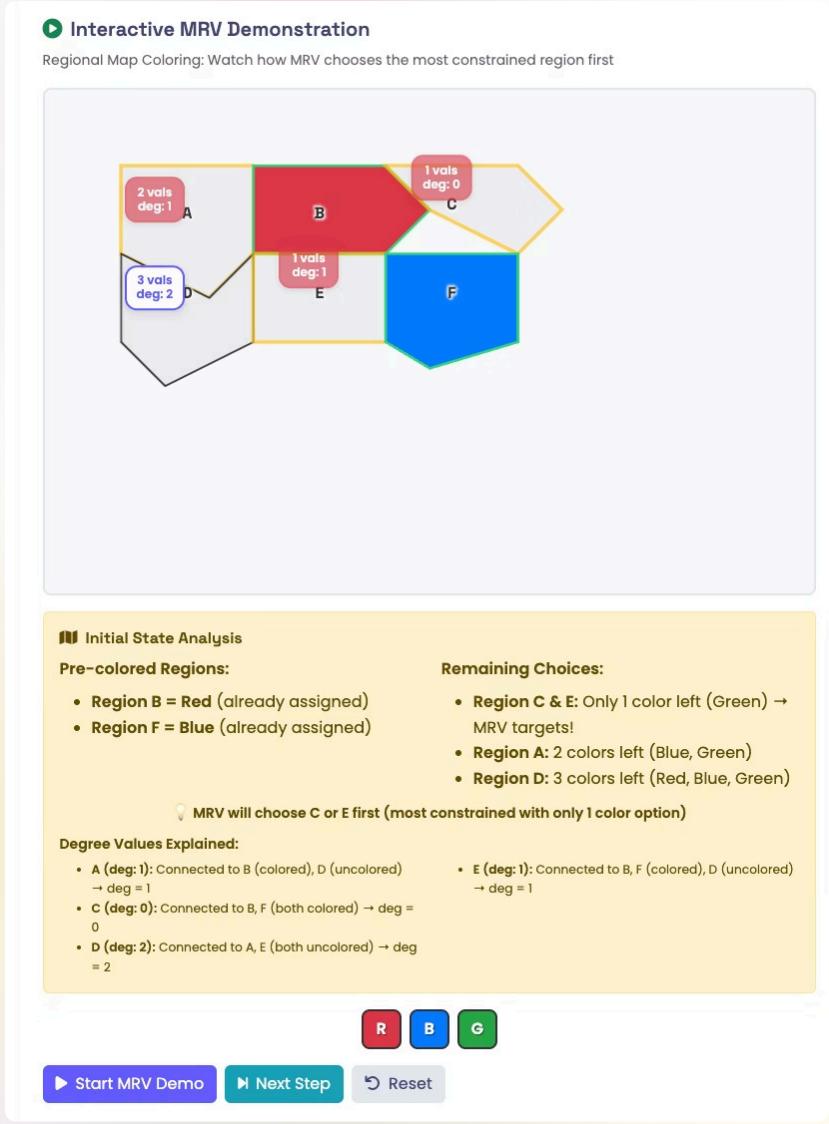
This approach **reduces the branching factor** by prioritizing variables that are most likely to cause **backtracking**.

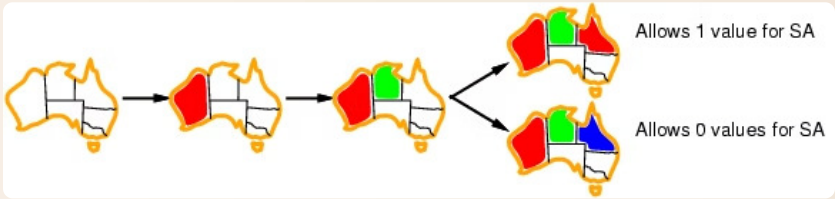
Strategic Advantage

Idea: Reduce the branching factor now

- Smallest domain size
- Fewest number of children
- Least branching in search tree

By choosing **constrained variables first**, we detect **failures early** and avoid wasting time on doomed partial assignments.





Least Constraining Value

Value Selection Strategy

Once we've chosen a variable (using MRV), we need to decide which value to assign first. The **least constraining value** heuristic guides this choice.

Selection Criterion

Choose the value that eliminates the **fewest values** from the domains of remaining unassigned variables.

Rationale

Preserve maximum flexibility for future variable assignments. The more options available later, the more likely we are to find a complete solution.

Implementation

For each possible value, count how many domain values would be eliminated from neighboring variables. Choose the value with the minimum count.

This heuristic is particularly effective when combined with MRV - we fail fast on variables but preserve options for solution finding.

END of Lecture

Systematic Search with Constraint Propagation

We'll trace through the 4-Queens problem using backtracking search enhanced with forward checking. This demonstrates how constraint propagation reduces the search space by eliminating invalid values early.

Problem Setup

4 variables Q_1, Q_2, Q_3, Q_4 representing queen positions in columns 1-4. Each has domain {1, 2, 3, 4} representing row positions.

Constraints

No two queens can share the same row, column, or diagonal: $Q_i \neq Q_j$ and $|Q_i - Q_j| \neq |i - j|$

Forward Checking

After each assignment, remove conflicting values from unassigned variables' domains



4-Queens: Initial State

Starting Configuration

All four variables Q_1, Q_2, Q_3, Q_4 have their full domains available.

4

Variables

Q_1, Q_2, Q_3, Q_4 (columns)

4

Domain Size

Each queen can be in rows 1-4

0

Assignments

No queens placed yet

0

Eliminations

No constraints applied yet

The search will begin by selecting the first variable Q_1 and trying different row positions. Red coloring in upcoming slides indicates assigned values, while eliminated values will be marked with X.



4-Queens: $Q_1 = 1$ Assignment

First Assignment and Constraint Propagation

Assign $Q_1 = 1$ (queen in column 1, row 1) and apply forward checking to eliminate conflicting values from remaining variables.

Row Conflicts

Q_2, Q_3, Q_4 cannot be in row 1

Diagonal Conflicts

Eliminate positions on diagonals from Q_1

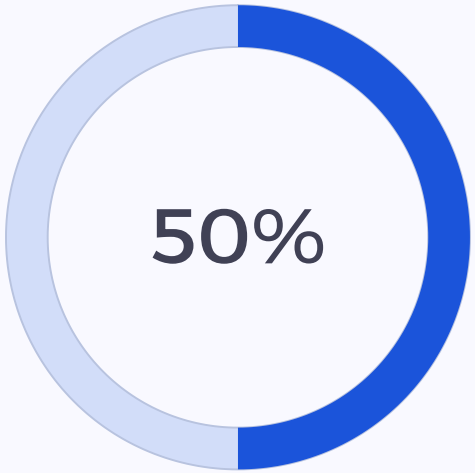
Eliminated values: $\{(Q_2,1), (Q_2,2), (Q_3,1), (Q_3,3), (Q_4,1), (Q_4,4)\}$

This eliminates 6 values from the domains of unassigned variables, significantly reducing the search space.

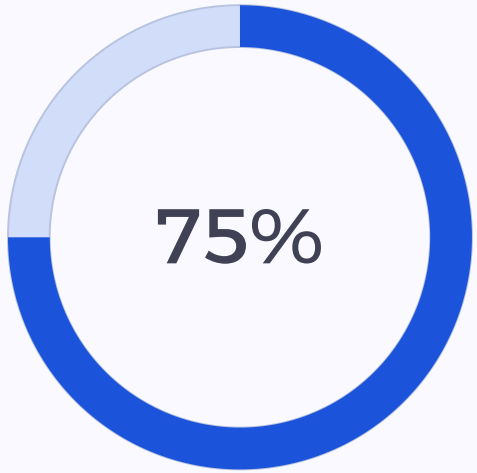
4-Queens: $Q_2 = 3$ Assignment

Second Assignment

With $Q_1 = 1$ already placed, try $Q_2 = 3$. This places the second queen in column 2, row 3.



Variables assigned



Domain values eliminated

After this assignment, apply forward checking to update domains of Q_3 and Q_4 by removing values that would conflict with both $Q_1 = 1$ and $Q_2 = 3$.

The red coloring indicates confirmed assignments, while the search space continues to shrink through constraint propagation.

4-Queens: Constraint Propagation Analysis

Cumulative Effects of Forward Checking

After $Q_1 = 1$ and $Q_2 = 3$, analyze the remaining domains:

Q_3 Available Values

- ~~Row 1:~~ Conflicts with Q_1
- ~~Row 2:~~ Diagonal conflict with Q_2
- ~~Row 3:~~ Conflicts with Q_2
- ~~Row 4:~~ Diagonal conflict with Q_2

Result: Q_3 domain is empty!

Q_4 Available Values

- ~~Row 1:~~ Conflicts with Q_1
- Row 2: Available
- ~~Row 3:~~ Diagonal conflict
- ~~Row 4:~~ Conflicts with Q_1 diagonal

Result: $Q_4 = \{2\}$ only

Forward checking detects that Q_3 has no legal values, triggering immediate backtrack. This saves exploring the rest of this branch.





4-Queens: Backtrack from $Q_2 = 3$

Failure Detection and Recovery

The assignment $Q_1 = 1$, $Q_2 = 3$ leads to failure because Q_3 has no legal values. Backtrack to try $Q_2 = 4$.

1. **Failure Recognition**
   Forward checking detects empty domain for Q_3
2. **Restore Domains**
   Undo constraint propagation from $Q_2 = 3$
3. **Try Next Value**
   Attempt $Q_2 = 4$ while keeping $Q_1 = 1$

This backtracking process continues systematically, trying all possible values while using forward checking to prune impossible branches early.



4-Queens: Cumulative Eliminations

Tracking Constraint Propagation

As we progress through the search, we accumulate eliminated values at each level:

Q_1 = 1 Level

Eliminated: {(Q_2,1), (Q_2,2), (Q_3,1), (Q_3,3), (Q_4,1), (Q_4,4)}

Six values removed due to row and diagonal conflicts

Q_2 = 3 Level

Additional eliminations: {(Q_3,2), (Q_3,4), (Q_4,3)}

Three more values eliminated, but Q_3 domain becomes empty

Early termination: We could fail as soon as Q_3's domain became empty after eliminating {(Q_3,2), (Q_3,4)}. No need to continue eliminating (Q_4,3).



4-Queens: $Q_2 = 4$ Attempt

Exploring Alternative Values

After backtracking from $Q_2 = 3$, try $Q_2 = 4$ with $Q_1 = 1$ still assigned.

Domain Restoration

Restore eliminated values from the failed $Q_2 = 3$ branch

New Assignment

Place second queen at $Q_2 = 4$ (column 2, row 4)

Forward Checking

Apply constraint propagation for the new assignment

The systematic search continues, maintaining the invariant that $Q_1 = 1$ while exploring all possible values for Q_2 before eventually backtracking to try different values for Q_1.

4-Queens: Systematic Failure Analysis

Why $Q_1 = 1$ Branch Fails

Through systematic exploration with forward checking, we discover that **all** values for Q_2 lead to failure when $Q_1 = 1$.



25%

$Q_2 = 1$: Row conflict



25%

$Q_2 = 2$: Diagonal conflict



25%

$Q_2 = 3$: Over-constrains Q_3



25%

$Q_2 = 4$: Creates conflicts

This exhaustive failure at the Q_2 level forces backtracking to Q_1, where we must restore all eliminated values and try $Q_1 = 2$. Forward checking dramatically reduced the work needed to discover this fundamental incompatibility.

4-Queens: $Q_1 = 2$ New Branch

Starting a New Search Branch

After failing with $Q_1 = 1$, backtrack completely and try $Q_1 = 2$. This creates a fresh starting point with different constraint propagation patterns.

Complete Backtrack

Restore all eliminated values from $Q_1 = 1$ branch

Fresh Assignment

Place first queen at $Q_1 = 2$ (column 1, row 2)

New Constraints

Apply forward checking with different elimination pattern

This new branch will have different constraint propagation effects, potentially leading to a successful solution or revealing new patterns of failure.



4-Queens: $Q_1 = 2$ Constraint Effects

Forward Checking from $Q_1 = 2$

With $Q_1 = 2$, apply forward checking to eliminate conflicting values from Q_2, Q_3, and Q_4.

Row 2 Conflicts

Q_2, Q_3, Q_4 cannot be placed in row 2 (same row as Q_1)

Diagonal Conflicts

Eliminate positions on both diagonals extending from (1,2)

Domain Updates

Systematically reduce domains of unassigned variables

New eliminations: $\{(Q_2,1), (Q_2,2), (Q_2,3), (Q_3,2), (Q_3,4), (Q_4,2)\}$

Notice the different pattern compared to $Q_1 = 1$, showing how the starting position fundamentally affects the constraint landscape.

4-Queens: $Q_2 = 4$ Under New Constraints

Continued Search with $Q_1 = 2$

With $Q_1 = 2$ eliminating several values, Q_2 has only one remaining option: $Q_2 = 4$.

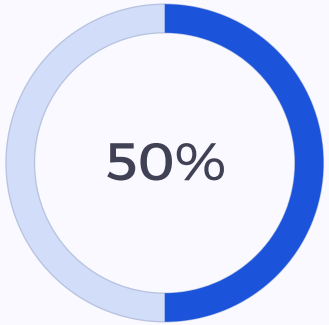
Forced Assignment

After $Q_1 = 2$ eliminates values $\{1, 2, 3\}$ from Q_2's domain, only $Q_2 = 4$ remains legal. This is a **unit constraint** - a forced move.

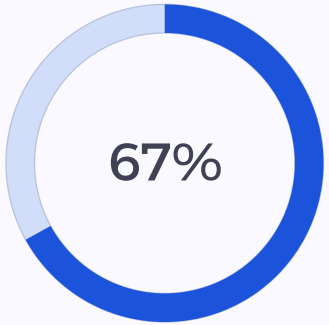
Next Constraint Propagation

Assign $Q_2 = 4$ and apply forward checking to Q_3 and Q_4:

Additional eliminations: $\{(Q_3,3), (Q_4,4)\}$



Variables assigned



Total eliminations

The search space continues to contract as constraint propagation eliminates impossible combinations systematically.



4-Queens: $Q_3 = 1$ Assignment

Third Queen Placement

With $Q_1 = 2$ and $Q_2 = 4$, examine the remaining domain for Q_3. After eliminations, Q_3 has domain {1, 3}. Try $Q_3 = 1$ first.

Domain Analysis

Q_3 original domain {1,2,3,4} reduced to {1,3} after constraint propagation

Constraint Check

Verify $Q_3 = 1$ doesn't conflict with $Q_1 = 2$, $Q_2 = 4$

1

2

3

4

Value Selection

Choose $Q_3 = 1$ (could use least constraining value heuristic)

Forward Checking

Apply constraint propagation to Q_4's remaining domain



4-Queens: Final Constraint Propagation

Q_4 Domain Analysis

After $Q_1 = 2$, $Q_2 = 4$, $Q_3 = 1$, apply forward checking to determine Q_4's feasible values.

Cumulative Eliminations

From $Q_1 = 2$: eliminated $(Q_4,2)$
From $Q_2 = 4$: eliminated $(Q_4,4)$
From $Q_3 = 1$: eliminates $(Q_4,1)$

Final Domain

Q_4 original domain: {1,2,3,4}
After all eliminations: {3}
$Q_4 = 3$ is the only remaining legal value

This represents another unit constraint - Q_4 must equal 3. Let's verify this leads to a valid solution by checking that $Q_4 = 3$ doesn't create any new conflicts.

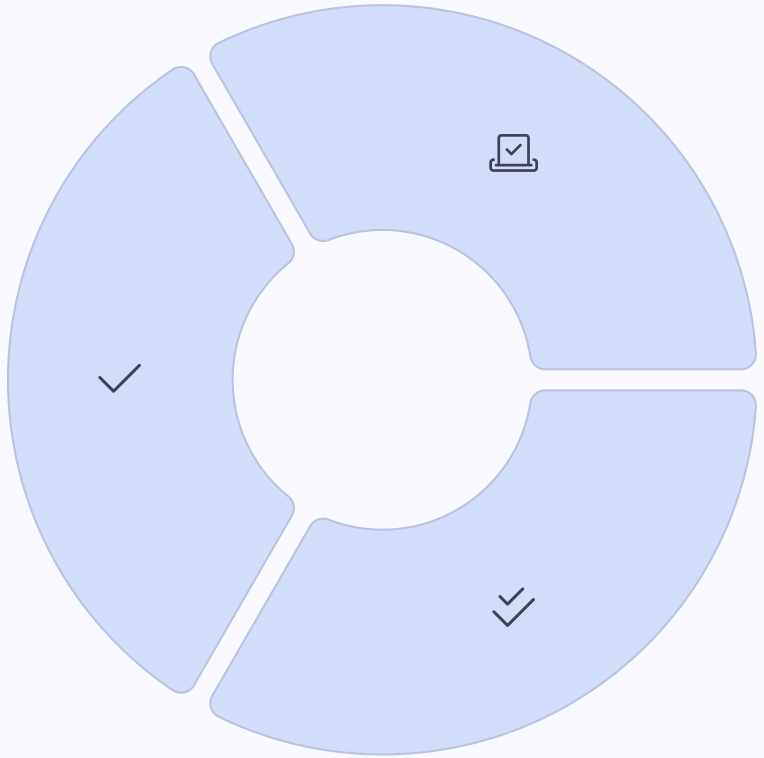
4-Queens: Solution Verification

Complete Assignment: $Q_1=2$, $Q_2=4$, $Q_3=1$, $Q_4=3$

Verify this assignment satisfies all constraints:

Row Constraints

All queens in different rows: {2,4,1,3} ✓



Column Constraints

Queens in different columns by construction ✓

Diagonal Constraints

No two queens share diagonals:

- $|2-4| \neq |1-2|$ ✓
- $|2-1| \neq |1-3|$ ✓
- $|2-3| \neq |1-4|$ ✓
- All other pairs checked ✓

SUCCESS! Forward checking guided us to a valid solution efficiently by eliminating impossible values throughout the search process.

4-Queens: Search Efficiency Analysis

Forward Checking Performance Benefits

Compare the search efficiency with and without forward checking:

16

Naive Backtracking

Complete search tree leaves (4^4 combinations)

8

With Forward Checking

Actual assignments explored before finding solution

50%

Search Reduction

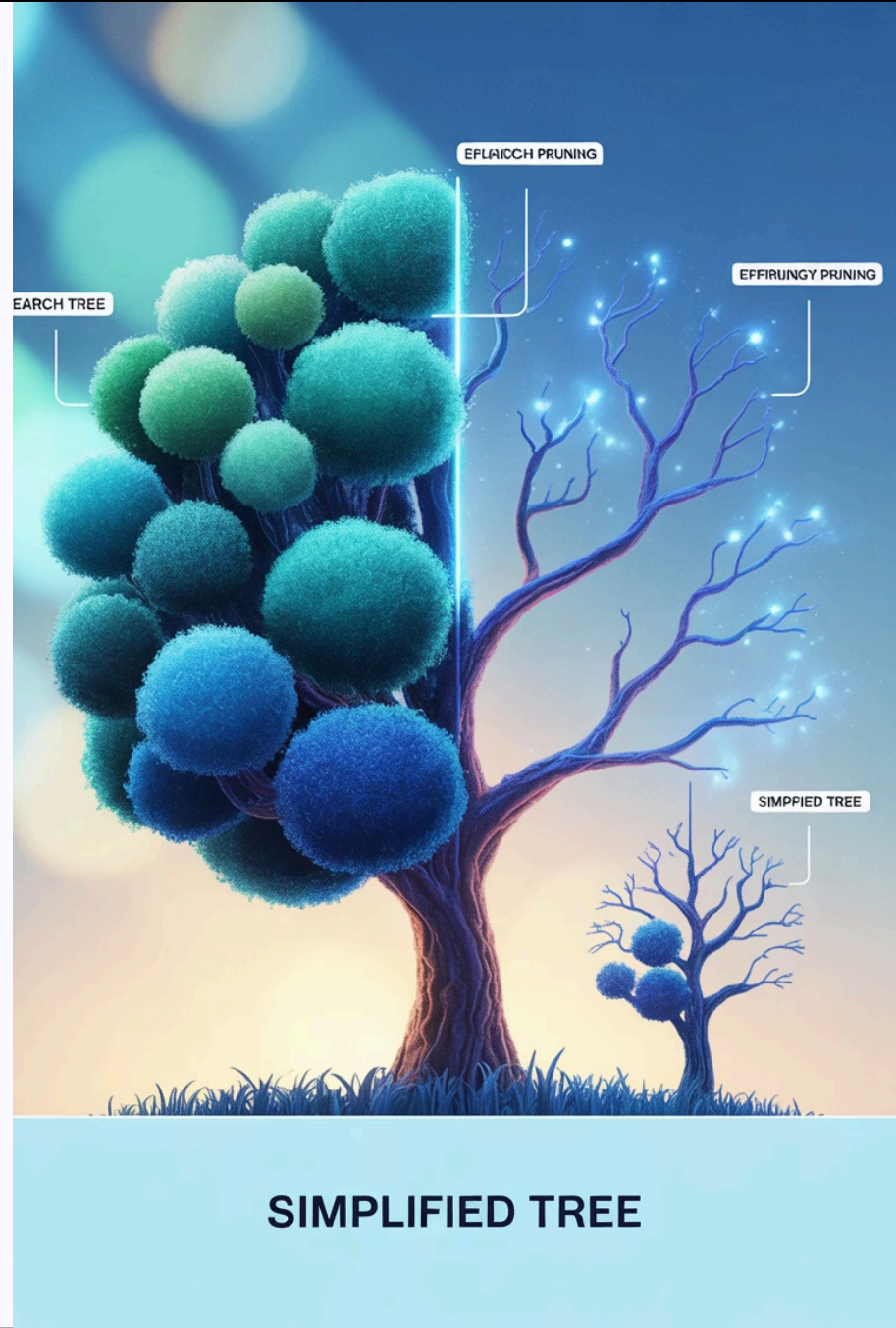
Eliminated through constraint propagation

90%

Failure Detection

Branches pruned by early domain elimination

Forward checking transforms an exponential search into a manageable systematic exploration by detecting failures early and maintaining arc consistency information throughout the search process.



4-Queens: Level-by-Level Domain Tracking

Systematic Domain Management

Track how domains change at each search level:



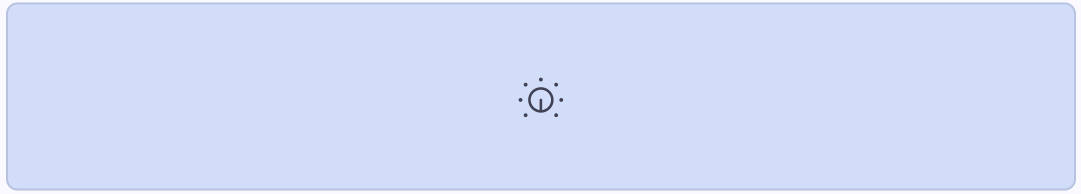
Q_1 = 2 Level

Eliminated: {(Q_2,1), (Q_2,2), (Q_2,3), (Q_3,2), (Q_3,4), (Q_4,2)}



Q_2 = 4 Level

Additional: {(Q_3,3), (Q_4,4)}



Q_3 = 1 Level

Final: {(Q_4,1)}

Each level builds upon previous constraint propagation, systematically reducing the solution space. The hierarchical domain tracking enables efficient backtracking when needed, restoring exactly the right values at each level.



4-Queens: Alternative Path Analysis

Exploring Different Branches

If $Q_3 = 1$ had failed, we would backtrack and try $Q_3 = 3$ (the only other value in its domain).

$Q_3 = 1$ Path

Led to successful solution $Q_4 = 3$

$Q_3 = 3$ Alternative

Would require different Q_4 analysis

Backtrack Strategy

Systematic exhaustion of value options

The beauty of backtracking with forward checking is its systematic guarantee: we explore exactly the necessary combinations while pruning impossible branches as early as possible.



4-Queens: Restoration Mechanisms

Domain Restoration Process

When backtracking occurs, we must carefully restore eliminated values to maintain search correctness.

Failure Recognition

Detect when a variable's domain becomes empty or no legal assignments exist

Level-Specific Restoration

Restore only the eliminations made at the current level, preserving higher-level constraints

Alternative Exploration

Try the next available value for the current variable

Complete Backtrack

If all values fail, backtrack to the previous variable level

For example, failing at $Q_3 = 2$ would restore $\{(Q_4,3)\}$ but maintain eliminations from Q_1 and Q_2 levels.

4-Queens: Final Solution Summary

Solution Found!

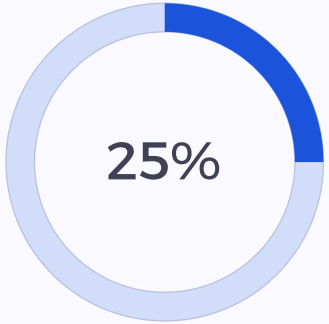
Winning Configuration

The systematic search with forward checking discovered:

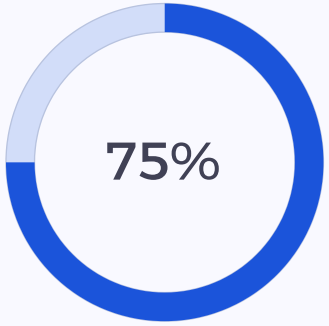
- **Q_1 = 2:** Column 1, Row 2
- **Q_2 = 4:** Column 2, Row 4
- **Q_3 = 1:** Column 3, Row 1
- **Q_4 = 3:** Column 4, Row 3

This assignment satisfies all constraints and represents a valid solution to the 4-Queens problem.

Search Efficiency



Search space explored



Branches pruned early

Forward checking reduced the search from 256 possible complete assignments (4^4) to examining only a small fraction through intelligent constraint propagation.



Forward Checking: Key Insights

Lessons from the 4-Queens Example

Early Failure Detection

Forward checking identifies dead-end branches immediately when variable domains become empty, avoiding wasted exploration of impossible solution paths.

Dynamic Domain Management

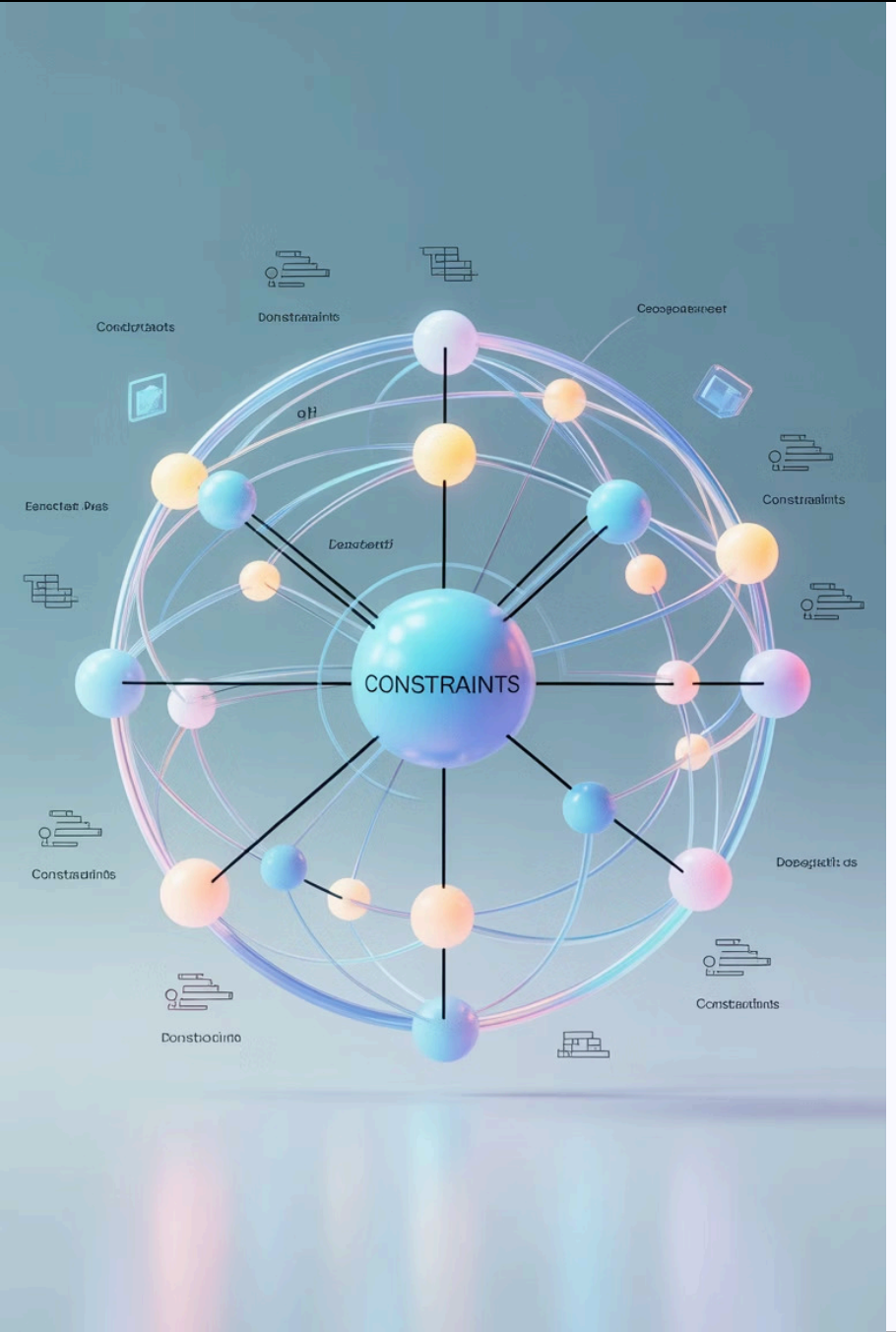
Maintaining current domain information for each variable enables the MRV heuristic and provides immediate feedback on constraint satisfaction.

Systematic Completeness

Despite pruning, forward checking maintains the completeness guarantee - if a solution exists, the algorithm will find it.

Precise Restoration

Careful tracking of which eliminations occur at each level enables exact restoration during backtracking without losing valid constraint propagation.



Beyond Forward Checking

Limitations and Extensions

While forward checking is powerful, it has limitations that motivate more advanced techniques:

Forward Checking Limitations

Only detects direct constraint violations between the assigned variable and unassigned neighbors. Cannot detect conflicts between unassigned variables that would result from current assignments.

Arc Consistency (AC)

More sophisticated constraint propagation that maintains consistency between all pairs of variables, detecting indirect conflicts that forward checking misses.

Maintaining Arc Consistency (MAC)

Applies full arc consistency after each assignment, providing maximum constraint propagation at the cost of increased computation per node.

These advanced techniques trade computational overhead for more aggressive pruning, often resulting in dramatic search space reductions for complex problems.

Local Search for CSPs: Alternative Approach

Moving Beyond Systematic Search

While backtracking with constraint propagation works well for many problems, some CSPs benefit from alternative approaches:

Iterative Improvement

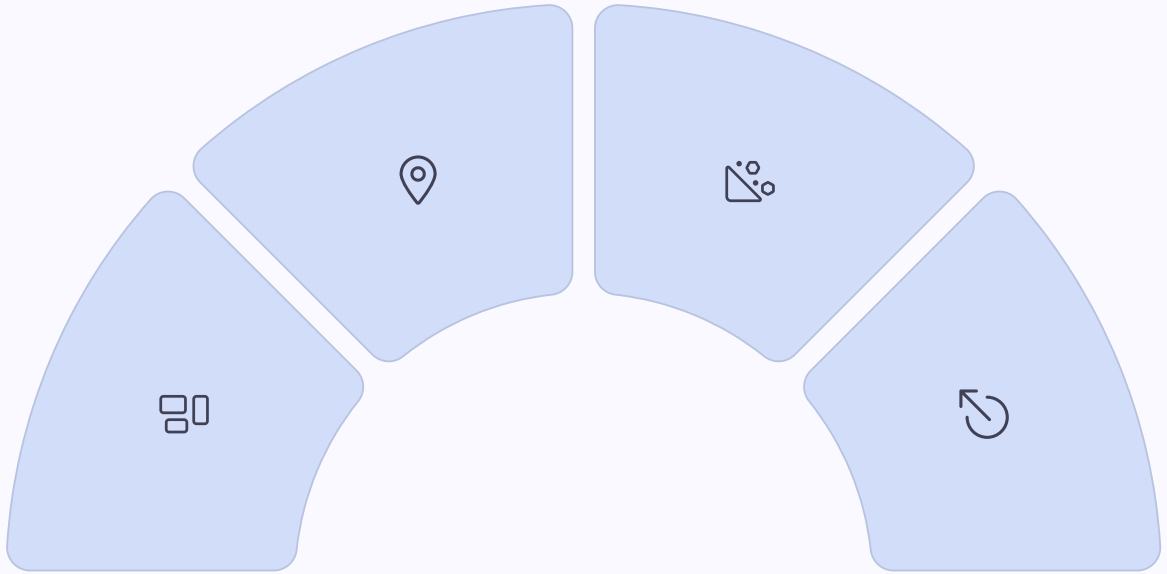
Make local changes to reduce constraint violations

Hill Climbing

Move toward assignments with fewer conflicts

Random Initialization

Start with a complete (possibly inconsistent) assignment



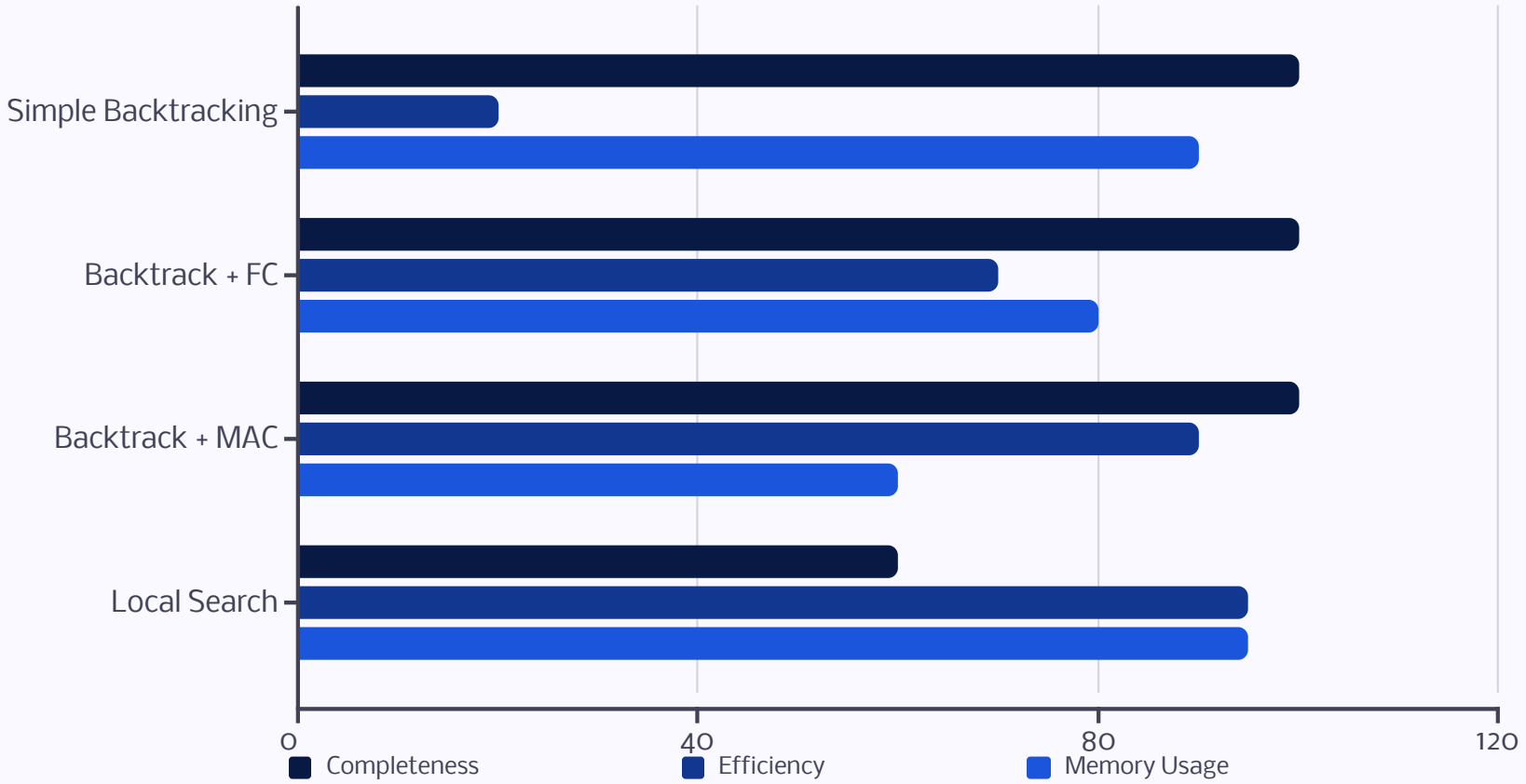
Escape Mechanisms

Techniques to avoid local minima (simulated annealing, etc.)

Local search is particularly effective for problems where finding any solution is more important than finding optimal solutions, and where the constraint graph has certain structural properties.

CSP Algorithm Comparison

Choosing the Right Approach



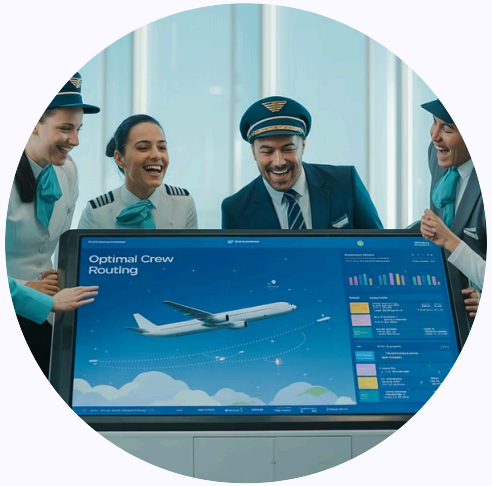
The choice depends on problem characteristics: systematic methods guarantee solutions but may be slow, while local search is fast but may not find solutions that exist.

Real-World CSP Success Stories



University Scheduling

Major universities use CSP techniques to schedule thousands of classes, considering room capacity, instructor availability, student conflicts, and resource constraints. Modern systems handle 50,000+ variables efficiently.



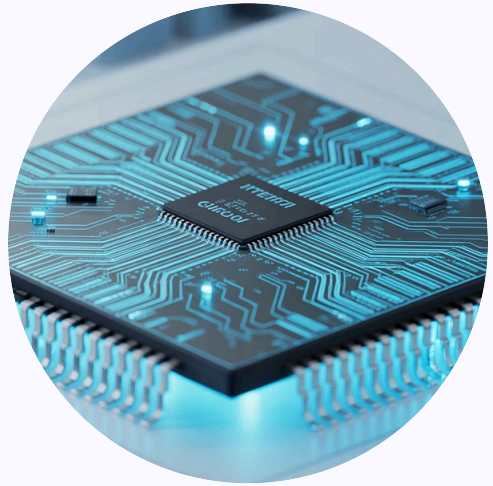
Airline Operations

Airlines optimize crew scheduling, gate assignments, and aircraft routing using CSPs. These systems save millions annually by reducing conflicts and maximizing resource utilization while satisfying regulatory requirements.



Manufacturing

Production scheduling in factories involves complex CSPs with temporal constraints, resource limitations, and precedence requirements. Advanced CSP solvers enable just-in-time manufacturing and minimize inventory costs.



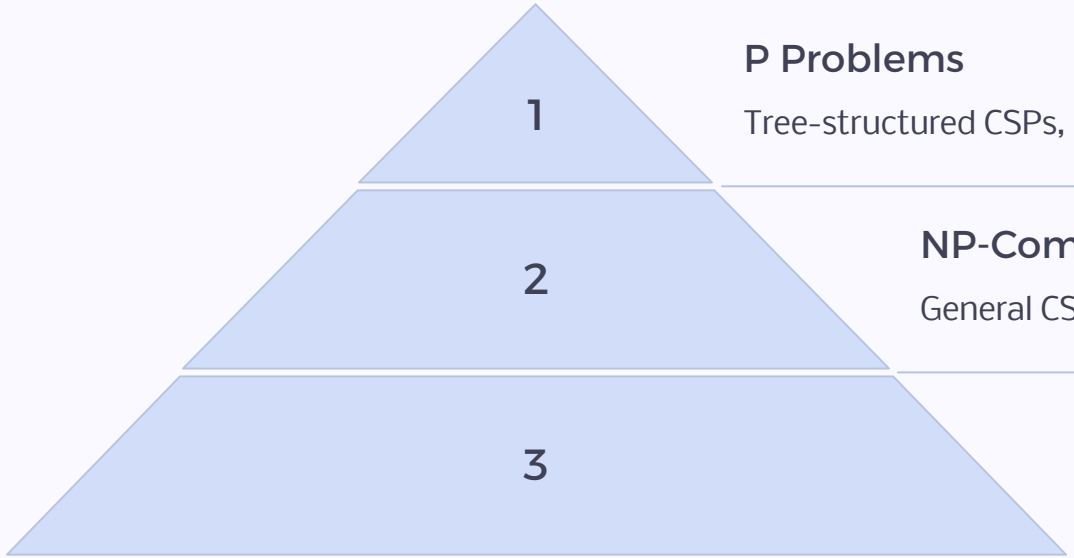
Circuit Design

VLSI chip design uses CSPs for component placement and wire routing, optimizing for area, timing, and manufacturability while satisfying electrical and physical constraints.

CSP Complexity and Tractability

Theoretical Foundations

Understanding when CSPs are difficult helps us choose appropriate solution methods:



P Problems

Tree-structured CSPs, bipartite matching

NP-Complete

General CSPs, graph coloring, Boolean satisfiability

Beyond NP

Quantified constraints, optimization versions

Structural Properties

Tree width, hypertree decomposition, and other graph-theoretic measures predict problem difficulty and guide algorithm selection.

Heuristic Performance

Even though CSPs are NP-complete in general, many real-world instances are solved efficiently by modern heuristic methods.

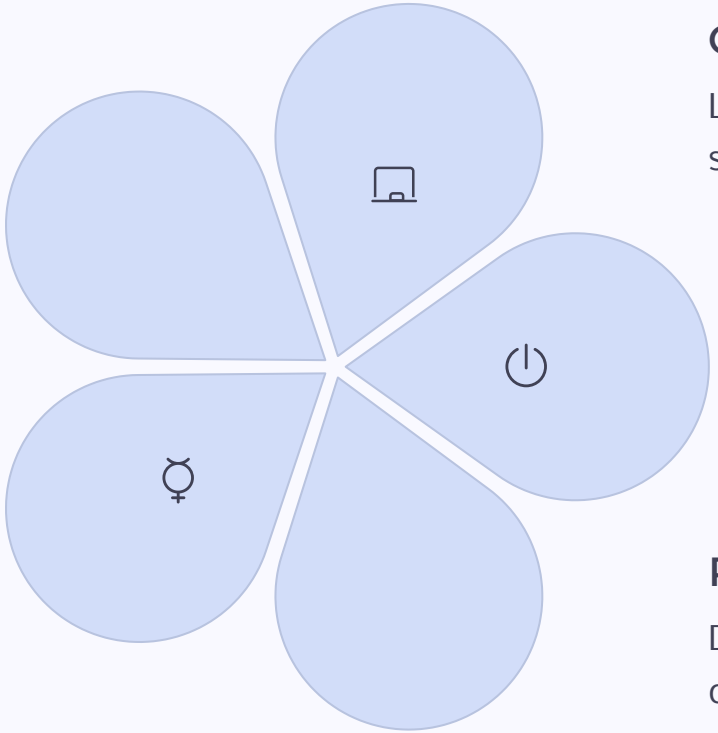
Advanced CSP Techniques

Problem Decomposition

Break large CSPs into smaller, manageable subproblems

Hybrid Approaches

Combine systematic and local search methods



Conflict-Directed Learning

Learn from failures to avoid repeating similar mistakes

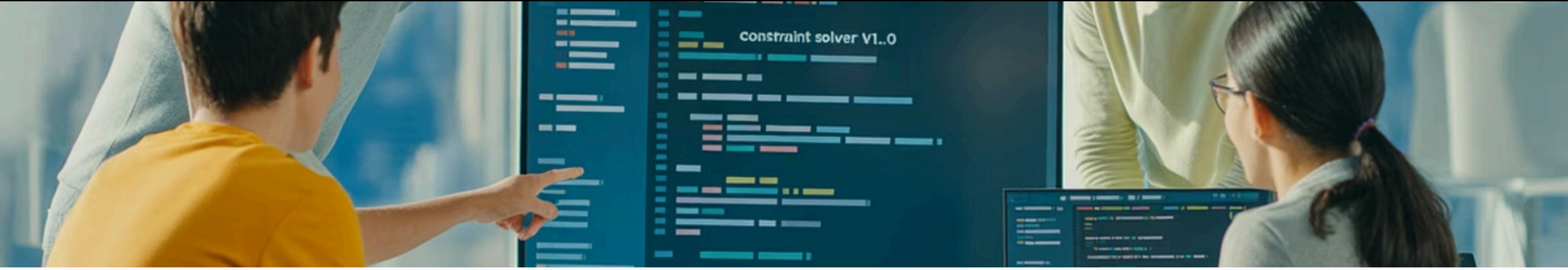
Restart Strategies

Periodically restart search with different variable orderings

Parallel Processing

Distribute search across multiple processors or cores

These advanced techniques push the boundaries of what CSP solvers can handle, enabling solutions to previously intractable problems in industry and research.



CSP Software and Tools

Practical Implementation Resources

01
10

Programming Libraries

Popular CSP libraries include Google OR-Tools, Choco Solver, Gecode, and SICStus Prolog. These provide high-level interfaces for modeling and solving CSPs efficiently.

Modeling Languages

Specialized languages like MiniZinc, AMPL, and OPL allow declarative specification of CSPs, separating problem modeling from solution algorithms.

Commercial Solvers

Industrial-strength solvers like IBM CPLEX CP Optimizer and FICO Xpress provide enterprise-grade CSP solving with advanced features and support.

Modern CSP tools hide much of the algorithmic complexity while providing powerful modeling capabilities and efficient solving engines.



Constraint Programming Paradigms

Different Approaches to CSP Implementation

Procedural Approach

Implement backtracking and constraint propagation explicitly using traditional programming languages

Declarative Modeling

Specify constraints and let the solver determine the solution strategy automatically

Logic Programming

Use languages like Prolog where CSPs are expressed as logical rules and solved through unification

Each paradigm offers different trade-offs between control, expressiveness, and ease of use. Declarative approaches are often preferred for rapid prototyping and complex constraint modeling.

CSP Extensions and Variants

Constraint Optimization Problems (COPs)

Extend CSPs to find not just any solution, but optimal solutions according to an objective function. Combine satisfaction with optimization.

Probabilistic CSPs

Incorporate uncertainty in constraints or variable domains. Useful for real-world problems with incomplete information or noisy data.

Temporal CSPs

Add temporal reasoning to constraints, handling problems where time relationships between events matter, such as scheduling and planning.

Continuous CSPs

Extend to real-valued variables with continuous domains, requiring different solving techniques like interval arithmetic and numerical methods.





Machine Learning and CSPs

Emerging Intersections

Modern AI increasingly combines CSPs with machine learning techniques:

1. **Learning Heuristics**
   Use ML to learn better variable and value ordering heuristics from problem instances
2. **Constraint Acquisition**
   Automatically learn constraints from data or user behavior
3. **Solution Prediction**
   Train models to predict solution quality or feasibility
4. **Hybrid Systems**
   Integrate CSP reasoning with neural networks and other ML models

This integration promises more adaptive, intelligent constraint solving systems that can learn from experience and handle more complex, real-world problems.



CSP Research Frontiers

Quantum CSPs

Exploring quantum algorithms for constraint satisfaction, potentially offering exponential speedups for certain problem classes

Distributed CSPs

Solving constraints across multiple agents or locations with communication limitations and privacy requirements

Dynamic CSPs

Handling problems where constraints change over time, requiring efficient incremental solving and solution adaptation

Multi-Objective CSPs

Optimizing multiple conflicting objectives simultaneously, finding Pareto-optimal solution sets

These research directions address the growing complexity and scale of real-world constraint satisfaction problems in modern applications.



Building Your CSP Skills

Recommended Learning Path

Master the Fundamentals

Thoroughly understand backtracking, forward checking, and arc consistency through hands-on implementation

Solve Classic Problems

Implement solutions for N-Queens, Sudoku, map coloring, and scheduling problems to build intuition

Learn CSP Tools

Gain proficiency with modern CSP libraries and modeling languages for practical applications

Explore Advanced Topics

Study optimization variants, probabilistic constraints, and hybrid approaches for research or advanced applications

Start with simple problems and gradually increase complexity. Understanding why algorithms work is more important than memorizing implementation details.

Key Takeaways

CSP Fundamentals

CSPs provide a unified framework for modeling and solving constraint-based problems using variables, domains, and constraints

Search Enhancement

Heuristics like MRV and least constraining value dramatically improve backtracking performance

Constraint Propagation

Forward checking and arc consistency detect failures early and reduce search space

Practical Impact

CSP techniques solve real-world problems in scheduling, planning, design, and optimization

Future Directions

Integration with machine learning and quantum computing opens new possibilities for constraint solving

Constraint satisfaction provides both theoretical foundations and practical tools for tackling complex combinatorial problems across diverse domains.



Thank You

Questions?

Contact

Prof. Anis Koubaa
SE444 - Artificial Intelligence

Next Steps

Practice implementing CSP algorithms and explore real-world applications

Resources

Course materials, textbook chapters, and CSP software documentation

Continue exploring the fascinating world of constraint satisfaction and its applications in artificial intelligence!

جامعة الفيصل
Alfaisal University

كلية الهندسة والحوسبة المتقدمة
College of Engineering and Advanced Computing

Introduction to Artificial Intelligence (SE 444)

Lecture 4: Local Search and Genetic Algorithms

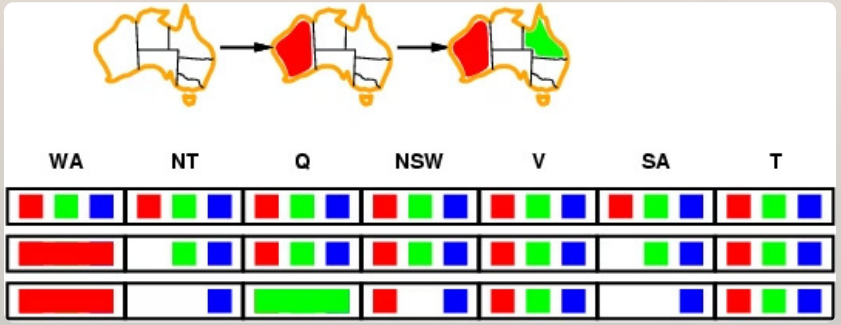
Prof. Anis Koubaa

akoubaa@alfaisal.edu

This lecture dives deep into the core of AI problem-solving, focusing specifically on **local search algorithms**. You'll uncover how AI agents navigate complex spaces to find optimal solutions using techniques such as **hill-climbing**, **simulated annealing**, and **genetic algorithms**. We will explore the strengths and weaknesses of each, equipping you with the fundamental skills to tackle real-world challenges through intelligent and adaptive search strategies.



Forward Checking - Step 2

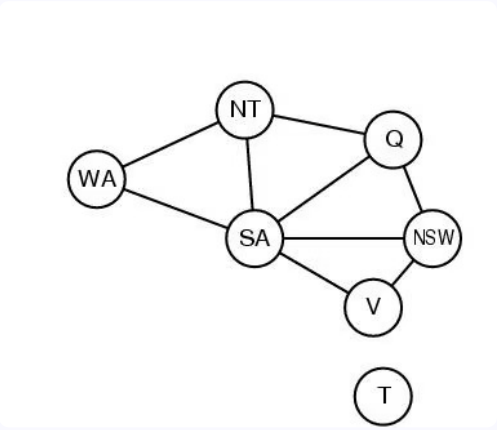


Assignment: Q = green

After WA = red, we assign Q = green. Now we must check Q's neighbors for constraint violations.

Impact on Q's neighbors:

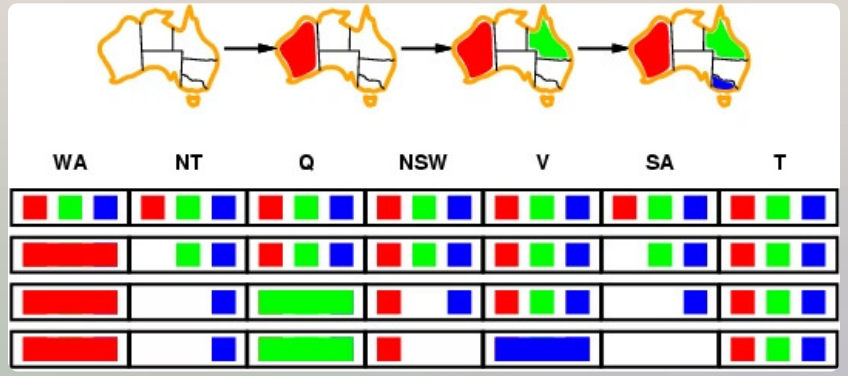
- **NT:** {green, blue} → {blue}
- **SA:** {green, blue} → {blue}
- **NSW:** {red, green, blue} → {red, blue}



Hidden Conflict

Notice that both NT and SA are now forced to be blue, but they're adjacent to each other! This creates an inevitable failure that forward checking doesn't immediately detect.

Forward checking is **local** - it only checks direct neighbors. More sophisticated constraint propagation techniques can detect this type of indirect conflict.



Forward Checking - Step 3

Assignment: V = blue

1. **V Assignment**

   Assign V = blue and check its neighbors: NSW and SA.

2. **Constraint Propagation**

   NSW: {red, blue} → {red}
   SA: {blue} → { } (empty!)

3. **Failure Detection**

   SA's domain becomes empty - no legal values remain. Forward checking detects this inconsistency immediately.

4. **Backtrack**

   The partial assignment is inconsistent with any complete assignment. Backtrack and try different values.

This demonstrates forward checking's power: it detected the failure as soon as SA's domain became empty, avoiding further futile search in this branch.